

Clase: Node.js, Vite y TypeScript (30 min)

Material de preparación para la clase. Incluye documentación de referencia, bosquejo de puntos con tiempos y libreto guía.

1. Documentación de referencia

Node.js

- **Sitio oficial:** <https://nodejs.org>
- **Docs API:** <https://nodejs.org/docs/latest/api/>
- **npm docs:** <https://docs.npmjs.com/>
- **Conceptos clave a manejar antes de la clase:**
 - Qué es un *runtime* de JavaScript (V8 fuera del navegador).
 - Modelo *event-driven, non-blocking I/O* (mencionarlo, no profundizar).
 - `package.json` como manifiesto del proyecto.
 - `node_modules` y por qué se ignora en `.gitignore`.
 - Diferencia entre `npm install` (todas las deps) y `npm install <paquete>`.
 - Scripts: `npm run dev`, `npm test`, etc.

TypeScript

- **Sitio oficial:** <https://www.typescriptlang.org/>
- **Handbook (la guía buena):** <https://www.typescriptlang.org/docs/handbook/intro.html>
- **Playground (útil para demo en vivo):** <https://www.typescriptlang.org/play>
- **Conceptos clave:**
 - Superset de JavaScript creado por Microsoft.
 - Tipado estático, se "transpila" a JavaScript.
 - `tsconfig.json` como configuración del compilador.
 - Tipos primitivos, `interface`, `type`, `enum`, genéricos (mostrar lo básico).
 - Errores en tiempo de compilación vs. tiempo de ejecución.

Vite

- **Sitio oficial:** <https://vite.dev/>
- **Guía:** <https://vite.dev/guide/>
- **¿Por qué Vite?:** <https://vite.dev/guide/why.html>

- **Conceptos clave:**
 - Build tool y dev server moderno (significa "rápido" en francés).
 - Creado por Evan You (también creador de Vue).
 - Usa ES modules nativos en desarrollo → arranque casi instantáneo.
 - Pre-bundling de dependencias con `esbuild`.
 - Build de producción con `Rollup`.
 - HMR (Hot Module Replacement) ultrarrápido.
 - Comparación con Webpack: Vite no rebundléa todo el proyecto en cada cambio.

Para mencionar como bonus (lo que comentó Mauro)

- **Pre-commit hooks:** `husky` + `lint-staged` para correr lint/tests antes de commitear.
- **Testing en Node:** `node:test` (test runner nativo desde Node 18+) o `vitest` (que se integra con Vite).

2. Bosquejo de puntos a dar

Distribución de los 30 minutos

Tiempo	Bloque	Modalidad
0:00 - 2:00	Introducción y objetivos	Slide / pizarrón
2:00 - 9:00	Node.js	Explicación + terminal
9:00 - 16:00	TypeScript	Explicación + ejemplo corto
16:00 - 26:00	Vite + demo integradora	Demo en vivo
26:00 - 28:00	Bonus: pre-commit y tests	Mención breve
28:00 - 30:00	Cierre y preguntas	Conversación

Bloque 1 — Introducción (2 min)

- Presentarse y decir el objetivo: "Al final de la clase van a entender qué hace cada herramienta y por qué se usan juntas".
- Mostrar el "para qué": estas tres tecnologías son la base del stack moderno de frontend que van a usar el resto de la carrera.
- Anticipar la demo final: "Vamos a crear un proyecto desde cero en vivo".

Bloque 2 — Node.js (7 min)

- **Qué es:** runtime de JavaScript fuera del navegador.
- **Por qué importa:** hasta ahora JS solo corría en el browser; Node permite usarlo para herramientas, scripts, servidores, etc.
- **npm:** el gestor de paquetes; ecosistema enorme.
- **package.json:** mostrar uno real; explicar `dependencies`, `devDependencies`, `scripts`.
- **Comandos esenciales:**
 - `node --version` / `npm --version`
 - `npm init -y`
 - `npm install <paquete>`
 - `npm run <script>`
- **Punto clave:** "Aunque hagamos frontend, vamos a usar Node todo el tiempo para correr las herramientas".

Bloque 3 — TypeScript (7 min)

- **Qué es:** JavaScript + tipos. Se compila a JS plano.
- **Problema que resuelve:** JS es muy permisivo, los errores aparecen recién en runtime.
- **Mostrar comparación rápida:**

```
js

// JavaScript
function sumar(a, b) { return a + b; }
sumar("2", 3); // "23" 
```

```
ts

// TypeScript
function sumar(a: number, b: number): number { return a + b; }
sumar("2", 3); // × Error en compilación
```

- **Ventajas concretas:**
 - El editor te avisa antes de correr el código.
 - Autocompletado real (IntelliSense).
 - Refactors más seguros.
 - Sirve como documentación viva.
- **Conceptos a mostrar:** tipos primitivos, `interface`, tipos de retorno, opcionales (`(?)`).

- **Punto clave (lo de Lucia):** "El tipado da trazabilidad: sabés qué espera y qué devuelve cada función sin tener que leerla entera".

Bloque 4 — Vite + Demo integradora (10 min)

- **Qué es:** dev server y build tool moderno.
- **Por qué se usa:** alternativa rápida a Webpack/CRA. Arranque casi instantáneo y HMR.
- **Demo en vivo (este es el corazón de la clase):**
 1. `npm create vite@latest mi-proyecto -- --template vanilla-ts`
 2. `cd mi-proyecto && npm install`
 3. Mostrar la estructura del proyecto en el editor.
 4. Abrir `package.json` y mostrar los scripts.
 5. `npm run dev` → abrir en el navegador.
 6. Editar un archivo `.ts`, guardar, mostrar el HMR.
 7. **Provocar un error de tipos a propósito** (pasar string donde espera number) → mostrar que el editor lo marca y que `npm run build` falla.
 8. Mostrar `npm run build` exitoso y la carpeta `dist/`.
- **Punto clave:** "Con un solo comando tenemos Node corriendo el server, Vite haciendo el bundle y TypeScript validando los tipos".

Bloque 5 — Bonus: pre-commit y tests (2 min)

- Mencionar que se puede ir más allá:
 - `husky` + `lint-staged`: corren validaciones antes de cada commit (no commitar código roto).
 - **Testing:** `node:test` viene con Node, o `vitest` que se integra naturalmente con Vite.
- No entrar en detalle, dejarlo como pista para que investiguen.

Bloque 6 — Cierre y preguntas (2 min)

- Resumen en una frase: "Node es el motor, TypeScript el control de calidad, Vite el armador del proyecto".
 - Recomendar como tarea: crear un proyecto Vite + TS y romperlo a propósito para ver cómo reacciona TS.
 - Preguntas.
-

3. Bosquejo de libreto

Cómo leerlo: las frases en cursiva son lo que decís en voz alta. Lo que está entre corchetes son acciones o cosas para mostrar. No es para leerlo literal, es una guía.

Apertura (0:00)

"Hola a todos, hoy vamos a ver tres herramientas que probablemente ya escucharon nombrar y que van a aparecer juntas durante toda la carrera: Node.js, TypeScript y Vite. La idea es que en 30 minutos entiendan qué hace cada una, y al final hacemos una demo creando un proyecto desde cero para ver cómo encajan."

[Mostrar slide con los tres logos]

*"Si tuviera que resumirlo en una frase: **Node es el motor, TypeScript es el control de calidad, y Vite es el armador del proyecto.** Vamos por partes."*

Node.js (2:00)

"Empecemos por Node. Hasta ahora JavaScript lo corrieron siempre dentro del navegador, ¿no? Bueno, en 2009 a alguien se le ocurrió tomar el motor de JavaScript de Chrome, que se llama V8, y meterlo en el sistema operativo para poder correr JS fuera del browser. Eso es Node.js."

*"¿Para qué nos sirve? Para todo lo que no es 'mostrar una página'. Servidores, scripts, herramientas, automatización... y, lo que más nos importa hoy, **para correr las herramientas con las que armamos el frontend.**"*

[Abrir terminal]

```
bash
node --version
npm --version
```

"Cuando instalan Node, viene con npm, que es el gestor de paquetes. npm es básicamente una biblioteca gigante de código que otra gente escribió y que ustedes pueden usar."

[Mostrar un `package.json` de ejemplo]

*"Este archivo, `package.json`, es el manifiesto del proyecto. Acá adentro está el nombre, las dependencias que necesita, y los **scripts** que se pueden correr con `npm run`. Cada vez que clonen un repo, lo primero que hacen es `npm install` para que se bajen todas las dependencias a `node_modules`. Y `node_modules` no se sube a Git, va siempre en el `.gitignore`."*

"Eso es lo mínimo que necesitan saber de Node por ahora. Vamos al siguiente."

TypeScript (9:00)

"TypeScript. ¿Alguien ya escribió algo en TypeScript? [esperar respuesta]. Bueno, lo más simple para entenderlo es: TypeScript es JavaScript con tipos. Lo creó Microsoft y básicamente lo que hace es compilarse a JavaScript plano antes de correr."

"¿Por qué se inventó? Porque JavaScript es muy permisivo. Miren esto:"

[Escribir o mostrar slide]

```
js

function sumar(a, b) {
  return a + b;
}

sumar("2", 3); // devuelve "23"
```

"En JavaScript esto no falla. Devuelve la string '23' y sigue como si nada. El error lo descubris cuando ya está en producción y un usuario te avisa."

"Ahora con TypeScript:"

```
ts

function sumar(a: number, b: number): number {
  return a + b;
}

sumar("2", 3); // × Error: el editor lo marca al instante
```

"Le decimos: 'a y b son números, y devuelvo un número! Y si alguien le pasa una string, **el editor lo marca antes de correr el programa, y la compilación falla**. Esto es lo que se llama tipado estático: chequea los tipos en tiempo de compilación, no en tiempo de ejecución."

"Las ventajas son tres principalmente:"

1. "Errores antes de correr — los ves en el editor mientras escribís."
2. "Autocompletado mucho mejor — el editor sabe qué métodos tiene cada cosa."
3. "Trazabilidad — leyendo la firma de la función ya sabés qué espera y qué devuelve, sin leer la implementación."

"Esa última es para mí la más importante en proyectos grandes: el tipado funciona como documentación viva."

Vite + Demo (16:00)

"Bueno, llegamos a la parte divertida. Vite. ¿Saben qué quiere decir 'vite' en francés? Rápido. Y es literal, lo van a ver."

"Vite es lo que se llama un build tool. Hace dos cosas: te corre un servidor de desarrollo mientras programás, y después te empaqueta todo para subirlo a producción. Lo creó Evan You, el mismo que hizo Vue."

"¿Por qué Vite y no Webpack o algo más viejo? Básicamente porque es muchísimo más rápido. Webpack tiene que bundlear todo el proyecto cada vez que cambia algo. Vite usa los módulos nativos del navegador en desarrollo y eso le permite arrancar casi al instante."

"Mejor que les explique con código. Vamos a crear un proyecto."

[Demo en vivo]

```
bash  
  
npm create vite@latest mi-proyecto -- --template vanilla-ts  
cd mi-proyecto  
npm install
```

"Fíjense lo que pasó. Le dije que use el template `vanilla-ts`, que significa TypeScript sin frameworks. Si abrimos el editor..."

[Abrir VS Code en la carpeta]

"Tenemos `index.html`, una carpeta `src` con TypeScript adentro, un `package.json` con scripts, un `tsconfig.json` con la config de TypeScript, y un `vite.config.ts` para configurar Vite. Todo armado por nosotros."

[Abrir `package.json`]

"Miren los scripts: `dev` para desarrollo, `build` para producción, `preview` para probar el build. Estos los corren con `npm run dev`, etc."

```
bash  
  
npm run dev
```

[Abrir el navegador en localhost:5173]

"Eso fue menos de un segundo. Ahora viene lo bueno."

[Editar `src/main.ts`, cambiar un texto]

"Sin recargar la página, el cambio aparece. Eso se llama Hot Module Replacement. Lo hace Vite por nosotros."

[Ahora forzar un error de tipos a propósito]

ts

```
function saludar(nombre: string): string {  
  return `Hola ${nombre}`;  
}  
  
saludar(42); // ✕
```

"Miren lo que pasa. El editor lo marca con el subrayado rojo. Y si intentamos hacer el build..."

bash

```
npm run build
```

"Falla. TypeScript nos protegió de subir código roto."

[Arreglar el error y volver a buildear]

bash

```
npm run build
```

"Ahora sí. Se generó la carpeta `dist/` con HTML, CSS y JavaScript optimizado, listo para subir a un servidor."

"Y todo esto en menos de 10 minutos, partiendo de cero."

Bonus (26:00)

"Antes de cerrar, dos cosas que les van a aparecer y que pueden investigar después:"

"Primero, **pre-commit hooks**. Con herramientas como `husky` y `lint-staged` se pueden configurar validaciones que se corren automáticamente antes de cada commit: lint, tests, formato. Si algo falla, el commit no se hace. Ayuda mucho a no romper la rama principal."

"Segundo, **tests**. Node trae desde la versión 18 un test runner integrado que se llama `node:test`. Y para proyectos con Vite hay uno que se llama `vitest` que se integra naturalmente. Para más adelante."

Cierre (28:00)

"Resumiendo en una frase: **Node es el motor que hace correr todo, TypeScript es el control de calidad que evita que metamos errores tontos, y Vite es el que arma el proyecto y lo empaqueta.** Las tres juntas son la base del desarrollo frontend moderno."

"Mi sugerencia: hagan lo que hicimos hoy en sus máquinas. Creen un proyecto Vite con TypeScript, escriban una función con tipos, rómpanla a propósito pasando algo del tipo equivocado, y vean cómo reacciona. Es la mejor forma de entenderlo."

"¿Preguntas?"

4. Tips para la presentación

- **Probá la demo antes.** Tener Node instalado, conexión a internet, y el comando `npm create vite` ya cacheado.
- **Tené un plan B.** Si falla la conexión, dejá un proyecto pre-creado en una carpeta para abrirlo directo.
- **No leas el libreto.** Es guía, no script. Mejor mirar a la clase.
- **Manejá los tiempos con un cronómetro discreto.** El bloque más sacrificable si te atrasás es el de bonus.
- **Si te sobra tiempo:** mostrá `tsconfig.json` y explicá una opción (`strict: true`) o mostrá un `interface` más complejo.
- **Si te falta tiempo:** saltate el bonus y andá directo al cierre.